

Video 2 — Codebase Walkthrough (~5:45)

Target: under 6 minutes. ~880 words, normal speaking pace.

Before recording: open `architecture.pdf` full-screen for the first 30 seconds, then switch to the editor.

Time	Show	Say this
0:00	The diagram (<code>architecture.pdf</code>)	"About 4,200 lines. The spine is one idea: derive everything once at build time, then let the browser do analytics instead of data-prep."
0:25	<code>scripts/build-dataset.mjs</code>	"The ETL. Papaparse reads the CSV, and for each row I derive distance from the hoop, the zone, the range band, the shot-clock and dribble buckets. Two compression tricks: every categorical gets dictionary-encoded to an integer, and I transpose rows into columns. That takes 1.6 megabytes of CSV to 666 kilobytes of JSON — about seven and a half times smaller than the same data as row objects."
1:00	Header comment, same file	"One deliberate omission: no input validation. The brief says the data is clean and asks for a simple solution, so the pipeline trusts it. I marked exactly where column, number and range guards would go if that assumption ever broke."
1:20	<code>lib/court.mjs</code>	"This is my favourite decision. Court geometry — the rim, the three-point arc, the eleven zones — lives in one plain <code>.mjs</code> file, and <i>both</i> the Node ETL and the React app import it. So the zone the pipeline classified a shot into and the polygon you see on screen are computed from the same constants. The picture literally cannot disagree with the numbers."
1:50	<code>data/decode.ts</code>	"Decode is the only module that understands the compact format. It unpacks columns and dictionaries back into plain objects once at load. Everything downstream reads normal <code>Shot</code> objects and never knows the wire format exists — so if I changed the encoding tomorrow, exactly one file changes."
2:15	<code>state/useFilters.ts</code>	"State is Zustand, and the rule is strict: the store holds filter <i>criteria</i> only, never derived data. No cached results, so nothing can go stale. Zustand over Context specifically because components subscribe to slices — with Context, toggling one filter would re-render thousands of SVG marks."
2:45	<code>state/useFilteredShots.ts</code>	"Derivation is a pure memoised function. It builds an array of predicates — only for filters that are actually active, so an empty filter set is nearly free — then returns two things: the filtered shots, and the team baseline, which is the same filters <i>minus</i> the player selection. That second one is what makes 'this player versus the team' honest: you're always comparing under identical context, never against an unfiltered season total."
3:20	<code>state/urlState.ts</code>	"Filters serialise straight to the query string. Only non-defaults get written, so a clean dashboard keeps a clean URL. It uses <code>replaceState</code> , not <code>pushState</code> — a filter rail is a dial, not a page, and pushing would put a history entry behind every single chip click."
3:45	<code>lib/metrics.ts</code> → <code>summarise()</code>	"Every aggregation funnels through <code>summarise</code> . Critically, rates are computed from raw counts, never averaged — so a total is the same however you slice it. eFG% is makes plus half your made threes over attempts, and points-per-shot is exactly twice that, in a unit coaches actually argue in."
4:15	<code>compareToBaseline / pointsAddedVsBaseline</code>	" <code>compareToBaseline</code> separates two different questions per zone: the eFG delta — is he better here — and the share delta — does he go here more often. Those are genuinely different findings. Someone plus eight in the corners while taking fewer of them is a play-design problem, not a shooting one. <code>pointsAddedVsBaseline</code> collapses all of it into one number: points above or below what the team would score on that same shot distribution."
4:50	<code>lib/colorScale.ts</code>	"One encoding vocabulary, built on d3 scales — a threshold scale for the diverging efficiency ramp, a quantize scale for volume, and a square-root scale for bubble radius so area, not radius, tracks the value. Kings purple drives the shell but appears in exactly one data role, the volume ramp, so brand never competes with meaning."
5:10	<code>components/court/ShotLayer.tsx</code>	"d3 does the maths, I render the marks. Hexbin for the honeycomb, and a quadtree here for hover — nearest-point in log-n instead of scanning every shot on every mouse move."
5:25	Filter rail, Assisted toggle	"Last thing — the bug worth telling. Filtering to 'assisted' gave me eFG over 100%. Not a math error: in the source, <code>assisted</code> is true on 2,586 makes and zero misses, because an assist is only credited on a made basket. So that filter was selecting makes only and forcing shooting percentage to a hundred. The fix was to filter on assist <i>opportunity</i> instead, which exists on makes and misses alike. The number was right — the question was wrong."
5:45	—	"That's the tour."

Accuracy notes

- **Don't say "there are no comments in the code."** There are file-level header comments in most files. The accurate — and stronger — claim is: "comments are kept to file-level headers explaining *why*, not line-by-line narration."
- **Spare-time story:** `court.ts` originally suppressed its `.mjs` import with a `@ts-expect-error` directive. It *looks* like a comment, so it got deleted in a cleanup pass and silently broke the build. Fixed properly with `allowJs`, which is strictly better — the declarations in `court.ts` are now verified against `court.mjs` instead of blindly asserted.
- **If asked about dead code:** the repo was swept — an orphaned `DataTable` component, unused `court.ts` re-exports, an unused `shotValue()`, and dead CSS were all removed. Bundle barely moved (80.16 → 80.13 KB gzipped) because Vite already tree-shook them; the cleanup was for the reader, not the user.